

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
(МОСКОВСКИЙ ПОЛИТЕХ)

КУРСОВОЙ ПРОЕКТ

по курсу
«Разработка веб-приложений»

ТЕМА

«Разработка веб-приложения для ведения семейного бюджета»

Выполнил: Момина Антонина Алексеевна

Группа: 241-327

Проверил: Кружалов Алексей Сергеевич

Москва, 2026

**«МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
(МОСКОВСКИЙ ПОЛИТЕХ)**

УТВЕРЖДАЮ
заведующая кафедрой
«Инфокогнитивные технологии»

_____ / Е. А. Пухова /

«_____» _____ 2026 г.

ЗАДАНИЕ

на выполнение курсовой работы (проекта)

Моминой Антонине Алексеевне,

обучающейся группы 241-327,

направления подготовки 09.03.01 «Информатика и вычислительная техника»

по дисциплине «Разработка веб-приложений»

на тему «**Разработка веб-приложения для ведения семейного бюджета**»

1. Исходные данные к работе (проекту): информационные ресурсы в сети интернет, научные публикации в открытой печати.

2. Содержание задания по курсовой работе (проекту) – перечень вопросов, подлежащих разработке:

Разрабатываемый вопрос	Объем, %	Срок	Примечание
Раздел 1. Анализ предметной области			
Задача 1.1. Обзор существующих программных продуктов по теме работы	10	22.03.2026	
Задача 1.2. Анализ программных инструментов разработки веб-приложений	7	26.03.2026	
Задача 1.3. Формулировка цели и задач работы	8	30.03.2026	
Раздел 2. Проектирование веб-приложения			
Задача 2.1. Анализ целевой аудитории	5	02.04.2026	
Задача 2.2. Описание функциональности приложения (use case, user story)	7	06.04.2026	

Разрабатываемый вопрос	Объем, %	Срок	Примечание
Задача 2.3. Проектирование модели данных (ER-диаграмма, логическая и физическая схемы БД)	8	10.04.2026	
Задача 2.4. Разработка макетов страниц (Wireframe)	5	14.04.2026	
Раздел 3. Разработка веб-приложения			
Задача 3.1. Разработка базовой структуры приложения и вёрстка шаблонов страниц	8	21.04.2026	
Задача 3.2. Реализация системы аутентификации и авторизации пользователей с ролями	6	27.04.2026	
Задача 3.3. Реализация CRUD-интерфейса для основных сущностей (Transaction, Category, Budget, Receipt)	7	04.05.2026	
Задача 3.4. Реализация обработки и анализа данных и загрузки/выгрузки файлов (чеки, отчёты)	12	12.05.2026	
Задача 3.5. Разработка системы статистики и визуализации (дашборды, диаграммы)	10	18.05.2026	
Раздел 4. Оформление итогов работы			
Задача 4.1. Создание Git-репозитория с кодом проекта	3	22.03.2026	
Задача 4.2. Деплой приложения на хостинг	4	26.05.2026	
Задача 4.3. Оформление отчёта о проделанной работе	3	26.05.2026	

Руководитель курсовой работы (проекта):
старший преподаватель кафедры «Инфокогнитивные технологии»

«____» _____ 2026 г. _____ А. С. Кружалов
(подпись)

Дата выдачи задания «1» апреля 2026 г.

Дата сдачи выполненной работы «13» июня 2026 г.

Задание принял к исполнению

«1» апреля 2026 г.  А. А. Момина
(подпись)

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	5
ГЛАВА 1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ	8
1.1 Обзор существующих программных продуктов	8
1.2 Анализ программных инструментов разработки	9
1.3 Формулировка цели и задач работы	11
ГЛАВА 2. ПРОЕКТИРОВАНИЕ ВЕБ-ПРИЛОЖЕНИЯ	13
2.1 Анализ целевой аудитории	13
2.2 Описание функциональности приложения	14
2.3 Проектирование модели данных	23
2.4 Разработка макетов страниц	25
ГЛАВА 3. РАЗРАБОТКА ВЕБ-ПРИЛОЖЕНИЯ	32
3.1 Вёрстка HTML страниц	32
3.2 Реализация модуля авторизации	33
3.3 Реализация CRUD-интерфейса	33
3.4 Реализация обработки и анализа данных	35
3.5 Реализация работы с файлами	38
3.6 Реализация системы визуализации	39
ЗАКЛЮЧЕНИЕ	42

ВВЕДЕНИЕ

В современных условиях эффективное управление личными и семейными финансами является важной задачей для большинства людей. Однако многие семьи сталкиваются с трудностями при контроле доходов и расходов.

Проблема. Основная проблема состоит в том, что у людей часто нет удобного и надёжного инструмента для систематического учёта финансов. Многие ведут записи в заметках или простых таблицах Excel. Это приводит к неконтролируемым тратам, невозможности правильно спланировать бюджет и снижению финансовой стабильности. Кроме того, существующие приложения часто неудобны для совместного использования несколькими людьми, имеют ограниченные возможности аналитики и не всегда позволяют работать с чеками и документами.

Актуальность. Тема учёта семейных финансов остаётся актуальной в 2026 году. Согласно исследованию Аналитического центра НАФИ и страховой компании «Росгосстрах Жизнь» (2025) [1], индекс финансовой грамотности россиян снизился до 12,61 балла из 21 возможного. Доля граждан с низким уровнем финансовой грамотности выросла до 34 %, а среди молодёжи 18–34 лет составляет 51 %. Почти половина работающих россиян регулярно испытывает финансовые трудности.

В условиях прогнозируемой инфляции 4,5–5,5 % (по данным Банка России на 2026 год) [2] семьи всё больше нуждаются в удобном инструменте для ведения общего семейного бюджета, отслеживания расходов всех членов семьи, загрузки чеков и получения наглядной статистики.

Степень разработанности проблемы. На сегодняшний день существует множество приложений для учёта финансов. Они позволяют вести учёт транзакций, ставить бюджеты и смотреть простые отчёты. Однако большинство из них имеют существенные недостатки: платные премиум-функции, слабая поддержка совместной работы нескольких пользователей, ограниченные возможности загрузки файлов и не всегда удобная аналитика. Готовых открытых веб-решений, которые одновременно включают систему ролей пользователей, полноценный CRUD-интерфейс, загрузку и выгрузку файлов чеков и глубокую агрегацию данных, относительно немного. Это определяет место и актуальность настоящей курсовой работы.

Объект исследования — процесс учёта и анализа доходов и расходов

семейного бюджета с помощью веб-технологий.

Предмет исследования — методы и средства разработки веб-приложения для управления личными финансами, включающего аутентификацию, авторизацию по ролям, CRUD-операции, агрегацию данных и работу с файлами.

Цель работы — разработать веб-приложение для учёта расходов и доходов семейного бюджета, которое обеспечит удобный совместный контроль финансов, аналитику и безопасность данных.

Для достижения поставленной цели необходимо решить следующие **задачи**:

- проанализировать существующие программные продукты и технологии разработки веб-приложений для управления финансами;
- определить требования целевой аудитории и спроектировать функциональность приложения (диаграммы вариантов использования, User Story);
- разработать модель данных (ER-диаграмму, логическую и физическую схемы базы данных);
- спроектировать пользовательский интерфейс (wireframes);
- реализовать систему аутентификации и авторизации с проверкой прав доступа по ролям;
- разработать CRUD-интерфейс для основных сущностей (транзакции, категории, бюджеты);
- реализовать механизмы загрузки и выгрузки файлов (чеки, отчёты) и агрегацию данных с визуализацией статистики;
- провести тестирование, деплой приложения и оформить отчётную документацию.

Методика исследования. В работе применялись методы системного анализа, сравнительный анализ существующих решений, структурное и объектно-ориентированное проектирование. Для реализации использовались современные веб-технологии (backend, frontend и база данных). Визуализация

данных выполнялась с помощью специализированных библиотек. Тестирование проводилось по функциональным и нефункциональным требованиям.

Структура отчёта соответствует логике поставленных задач и включает анализ предметной области, проектирование, описание реализации и заключение.

ГЛАВА 1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

1.1 Обзор существующих программных продуктов

На рынке представлено большое количество приложений для учёта личных и семейных расходов. Большинство из них — это мобильные приложения (Android/iOS). Полноценных веб-приложений, специально ориентированных на совместный семейный бюджет, на 2026 год очень мало.

Для сравнения были выбраны наиболее популярные решения: Дзен-мани [3], Monefy [4] и Money Lover [5]. В таблице ниже приведены их ключевые характеристики с точки зрения семейного использования.

Таблица 1.1 – Сравнение популярных приложений для учёта финансов

Характеристика	Дзен-мани	Monefy	Money Lover
Платформа	Мобильное	Мобильное	Мобильное + Web
Совместный бюджет	Полноценный	Сильно ограничен	Есть (в Premium)
Загрузка чеков	Есть (QR-сканирование)	Ограничено	Есть (фото)
Веб-версия	Ограниченная	Отсутствует	Полноценная
Стоимость ключевых функций	Премиум-подписка (многие отчёты и прогнозы)	Премиум для расширенной аналитики и синхронизации	Unlimited wallets и shared budgets — только Premium

Как видно из таблицы, у каждого рассмотренного приложения есть существенные недостатки именно при семейном использовании:

- **Дзен-мани** — лучший функционал совместного бюджета, но большинство важных отчётов, прогнозов и расширенной аналитики доступны только по платной подписке.
- **Monefy** — очень простой и интуитивный интерфейс, но совместный доступ сильно ограничен, а полноценной веб-версии нет.
- **Money Lover** — имеет веб-версию, однако создание нескольких общих кошельков и расширенные возможности бюджетирования также требуют платной подписки.

Таким образом, несмотря на обилие мобильных решений, на рынке практически отсутствует удобное, бесплатное (или полностью функциональное в базовой версии) веб-приложение, которое одновременно предоставляет:

- полноценную совместную работу нескольких пользователей,
- систему ролей и разграничения прав доступа,
- удобную загрузку и выгрузку файлов (чеков, отчётов),
- глубокую агрегацию данных и наглядную статистику,
- доступ с любого устройства без установки приложений.

Вывод. Разработка собственного веб-приложения для учёта семейных расходов позволит устранить указанные недостатки существующих решений и создать удобный, бесплатный в базовой функциональности инструмент, специально ориентированный на совместное использование всей семьёй.

1.2 Анализ программных инструментов разработки

Для реализации веб-приложения были рассмотрены популярные технологии и стеки разработки. Особое внимание уделялось простоте освоения, скорости разработки, наличию необходимого функционала (аутентификация, работа с базой данных, загрузка файлов, визуализация) и подходящему уровню сложности для курсового проекта.

Наиболее распространёнными Python-фреймворками для создания веб-приложений в 2026 году являются Django [6], Flask [7] и FastAPI [8].

Django — мощный full-stack фреймворк, включающий в себя большое количество готовых компонентов («батарейки из коробки»): встроенную систему аутентификации, ORM, админ-панель и механизмы защиты. Главное преимущество — высокая скорость разработки и хорошая безопасность. Основной недостаток — избыточная сложность и «тяжеловесность» для небольшого учебного проекта.

FastAPI — современный высокопроизводительный асинхронный фреймворк для создания API. Отличается отличной поддержкой типизации и автоматической генерацией документации. Преимущества: высокая скорость работы. Недостатки: избыточен для классического CRUD-приложения с серверным рендерингом шаблонов.

Flask — лёгкий микрофреймворк, который предоставляет только базовый функционал. Все необходимые расширения подключаются по мере

необходимости. Главные преимущества — высокая гибкость, минимализм, простота изучения и отличная читаемость кода.

Для управления базой данных рассматривались две основные СУБД: SQLite и PostgreSQL.

SQLite — простая файловая база данных, не требующая установки сервера. Преимущества: крайне простая настройка, нулевые затраты на развертывание, высокая скорость для небольших проектов. Недостатки: слабая поддержка нескольких пользователей одновременно, ограниченные возможности по масштабированию и ограниченный функционал.

PostgreSQL — полноценная клиент-серверная реляционная СУБД. Преимущества: высокая производительность при работе с несколькими пользователями, возможности сложных запросов, хорошая масштабируемость и надёжность. Недостатки: более сложная настройка по сравнению с SQLite.

Для создания пользовательского интерфейса сравнивались Bootstrap 5 [9] и полноценные JavaScript-фреймворки React [10] и Vue.

Bootstrap 5 — CSS-фреймворк с готовыми компонентами и встроенной адаптивностью. Преимущества: очень быстрая вёрстка, минимальный порог вхождения, удобен для адаптивности. Недостатки: менее гибкий при создании сложной интерактивной логики.

React и **Vue** — мощные JavaScript-фреймворки для создания веб-приложений. Преимущества: высокая интерактивность, компонентный подход, отличная производительность при большом объёме данных. Недостатки: значительно более сложные в использовании, необходимость в дополнительных инструментах (Vite, Redux/Vuex и т.д.), что избыточно для курсового проекта.

После проведённого анализа был выбран следующий стек разработки:

- **Backend:** Python 3.12 + Flask 3.1
- **ORM:** Flask-SQLAlchemy
- **Аутентификация и авторизация:** Flask-Login + Flask-Bcrypt
- **База данных:** PostgreSQL
- **Frontend:** HTML5, CSS3, JavaScript, Bootstrap 5.3
- **Визуализация данных:** Chart.js [11]

- **Дополнительные инструменты:** Git, Visual Studio Code

Данный стек обеспечивает оптимальный баланс между простотой реализации, читаемостью кода и требуемым функционалом. Flask позволяет глубоко понять принципы построения веб-приложений без избыточных абстракций, что особенно важно в рамках курсовой работы. PostgreSQL выбрана с целью приближения проекта к реальным условиям разработки.

1.3 Формулировка цели и задач работы

Цель работы — разработать веб-приложение для учёта расходов и доходов семейного бюджета, обеспечивающее удобный совместный контроль финансов, прозрачность трат и наглядную аналитику для всех членов семьи.

Для достижения поставленной цели необходимо решить следующие задачи:

1. Проанализировать существующие программные продукты и технологии разработки веб-приложений для управления семейными финансами.
2. Определить требования целевой аудитории и спроектировать функциональность приложения (диаграммы вариантов использования, User Story).
3. Разработать модель данных (ER-диаграмму, логическую и физическую схемы базы данных).
4. Создать макеты пользовательского интерфейса (wireframes).
5. Реализовать систему аутентификации и авторизации пользователей с разделением прав доступа по ролям.
6. Разработать CRUD-интерфейс для основных сущностей (транзакции, категории, бюджеты).
7. Реализовать механизмы загрузки и выгрузки файлов (чеки, отчёты) и систему агрегации данных с визуализацией статистики.
8. Провести тестирование приложения, выполнить деплой и оформить отчётную документацию.

В результате работы будет создано полноценное веб-приложение, соответствующее всем требованиям курсового проекта.

ГЛАВА 2. ПРОЕКТИРОВАНИЕ ВЕБ-ПРИЛОЖЕНИЯ

2.1 Анализ целевой аудитории

Для успешной разработки веб-приложения по ведению семейного бюджета необходимо чётко понимать, для кого оно создаётся, какие задачи будут решать пользователи и с какими ограничениями (техническими, финансовыми, временными) они сталкиваются в повседневной жизни.

На основе потребностей, технической подготовки и мотивации к ведению учёта была выделена одна, самая многочисленная и проблемная категории пользователей: семьи с совместным (полностью или частично) бюджетом.

2.1.1 Проблематика

Семьи с совместным бюджетом сталкиваются с рядом типичных проблем:

- Отсутствие финансовой прозрачности. Часто только один из партнёров имеет полное представление о доходах и расходах, что порождает недоверие.
- Споры о тратах и невозможность объективной оценки. При отсутствии единой записной книжки невозможно объективно сравнить расходы разных членов семьи.
- Сложность планирования крупных покупок. При обдумывании больших трат необходимо иметь актуальную картину по доходам и обязательным расходам.
- Необходимость контролировать разные типы расходов. Семейный бюджет неоднороден: есть общие траты и личные траты. Без разграничения можно незаметить лишние расходы.
- Разная мотивация к ведению учёта. Один партнёр может хотеть детальной аналитики, а второй лишь изредка вносить расходы. Инструмент должен учитывать оба типа поведения.

2.1.2 Потребности ЦА

На основе выявленных проблем были сформулированы следующие потребности, которые должно закрыть разрабатываемое приложение:

Таблица 2.1 – Потребности целевой аудитории

Потребность	Описание	Задача из ТЗ
Единое хранилище	Все транзакции семьи в одном месте, доступ с любого устройства	3.3 (CRUD)
Разделение по ролям	Администратор, родитель, ребенок	3.2 (авторизация с ролями)
Категоризация трат	Общие и личные категории, возможность создания/редактирования	3.3 (категории)
Агрегация по участникам	Автоматический подсчёт расходов каждого члена семьи за период	3.5 (аналитика)
Базовые отчёты	По периодам, категориям, участникам	3.5 (статистика)
Привязка чеков	Загрузка фото/скана чека к транзакции	3.4 (файлы)
Наглядная визуализация	Диаграммы и графики для быстрого понимания структуры расходов	3.5 (Chart.js)

2.1.3 Портрет типичного представителя категории

- Семейное положение: состоит в браке или зарегистрированном партнёрстве.
- Состав семьи: 2–4 человека (родители + 1–2 детей).
- Используемые устройства: смартфон, ноутбук или ПК.

2.1.4 Ключевые функции приложения

На основе анализа проблем и потребностей выделен основной набор функций:

Таблица 2.2 – Основные функции

Функция	Важность	Реализация
Аутентификация и разделение ролей	Критическая	Задача 3.2
CRUD для транзакций	Критическая	Задача 3.3
Фильтрация по дате, категории, участнику	Высокая	Задача 3.3
CRUD для категорий	Высокая	Задача 3.3
CRUD для бюджетов и лимитов	Высокая	Задача 3.3
Визуализация (диаграммы)	Высокая	Задача 3.5
Загрузка чеков	Средняя	Задача 3.4
Экспорт отчёта	Средняя	Задача 3.4

2.2 Описание функциональности приложения

Разрабатываемое веб-приложение должно предоставлять зарегистрированным пользователям возможность вести совместный учёт доходов и

расходов, управлять категориями и бюджетами, загружать чеки и просматривать наглядную статистику.

2.2.1 Диаграммы вариантов использования функциональных модулей

Для наглядного представления функциональных возможностей разрабатываемого веб-приложения были построены диаграммы вариантов использования (Use Case Diagram). Данные диаграммы позволяют описать взаимодействие акторов (пользователей системы) с системой и определить состав выполняемых ими операций.

В рамках проектирования были выделены четыре основных актора:

- **Неавторизованный пользователь** — любой посетитель, не прошедший аутентификацию;
- **Владелец семьи** — взрослый член семьи, отвечающий за ведение бюджета, управление категориями, лимитами и участниками;
- **Участник семьи** — младший член семьи с ограниченными правами;
- **Администратор (разработчик)** — технический администратор системы, отвечающий за обслуживание, резервное копирование и мониторинг.

Ниже представлено описание каждого функционального модуля: для каждого модуля приведена таблица действий акторов и диаграмма вариантов использования.

Модуль аутентификации и управления пользователями

Данный модуль отвечает за регистрацию, вход в систему и управление учётными записями пользователей. В таблице 2.3 приведено соответствие акторов и доступных им действий.

Таблица 2.3 – Действия акторов в модуле аутентификации и управления пользователями

Актор	Доступные действия
Неавторизованный пользователь	Регистрация, вход в систему
Родитель	Просмотр профиля, изменение пароля, выход из системы, добавление участника (ребёнка), удаление участника, настройка лимитов для участников
Ребёнок	Вход в систему, выход из системы, изменение пароля, просмотр своего профиля
Администратор	Просмотр всех пользователей, поиск пользователя, блокировка/разблокировка пользователя, удаление пользователя, сброс пароля пользователя

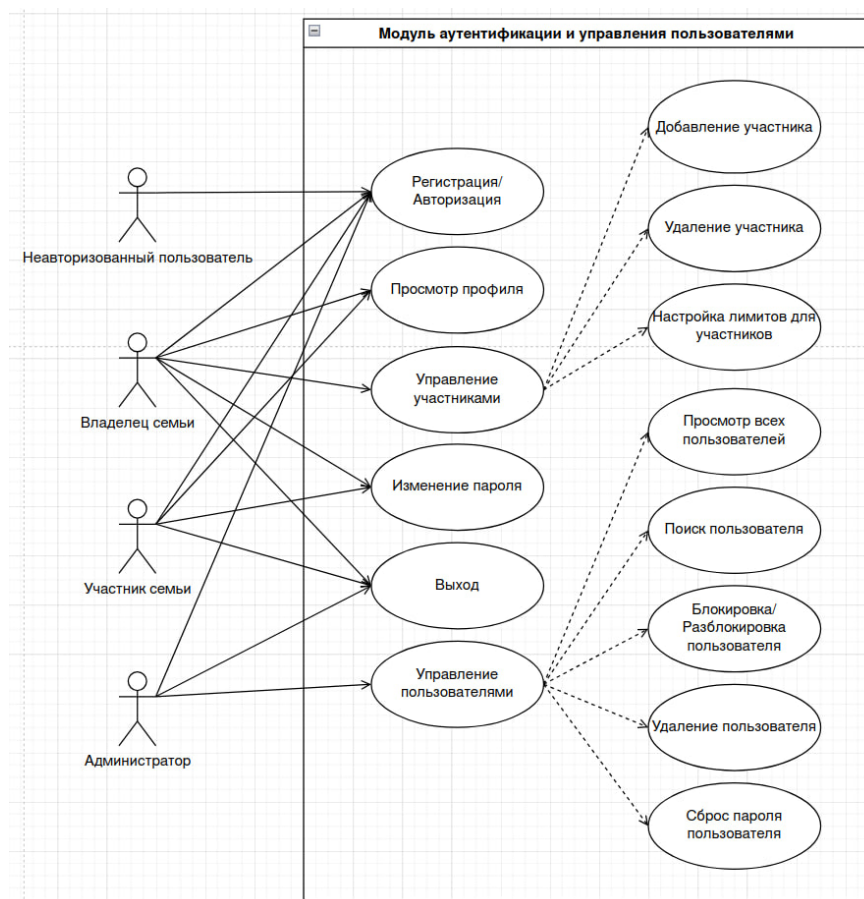


Рисунок 2.1 – Диаграмма вариантов использования модуля аутентификации и управления пользователями

Модуль управления транзакциями

Модуль управления транзакциями является ядром приложения. Он обеспечивает выполнение CRUD-операций (создание, чтение, обновление,

удаление) над доходами и расходами семьи. В таблице 2.4 приведено разграничение прав между родителем и ребёнком.

Таблица 2.4 – Действия акторов в модуле управления транзакциями

Актор	Доступные действия
Родитель	Просмотр всех транзакций семьи, добавление транзакции (доход/расход), редактирование любой транзакции, удаление любой транзакции, фильтрация транзакций (по дате, категории, участнику), сортировка транзакций
Ребёнок	Просмотр своих транзакций, добавление расхода (только свои), редактирование своей транзакции, удаление своей транзакции, фильтрация своих транзакций

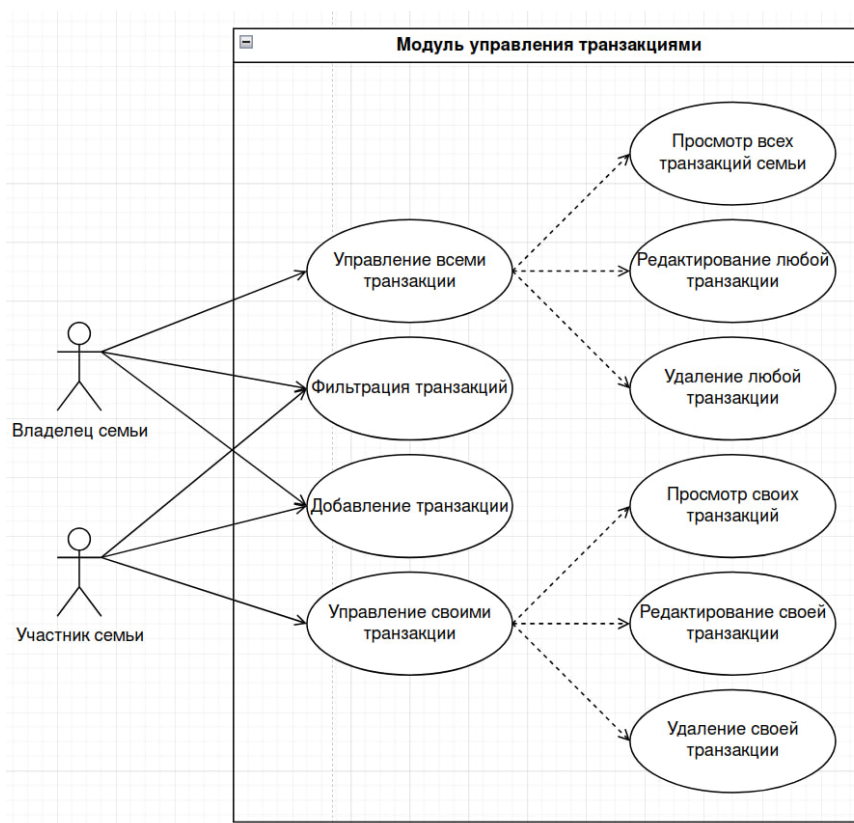


Рисунок 2.2 – Диаграмма вариантов использования модуля управления транзакциями

Модуль управления категориями и бюджетами

Данный модуль позволяет структурировать расходы и доходы по категориям, а также устанавливать бюджетные лимиты для контроля трат. Родитель может гибко настраивать категории и лимиты, ребёнок имеет только права на просмотр. В таблице 2.5 описаны действия акторов.

Таблица 2.5 – Действия акторов в модуле управления категориями и бюджетами

Актор	Доступные действия
Родитель	Просмотр категорий, добавление категории, редактирование категории, удаление категории, установка лимита на категорию, удаление лимита на категорию, просмотр исполнения бюджетов, установка бюджета (лимита карманных денег) для участника
Ребёнок	Просмотр исполнения бюджетов (только своих лимитов)

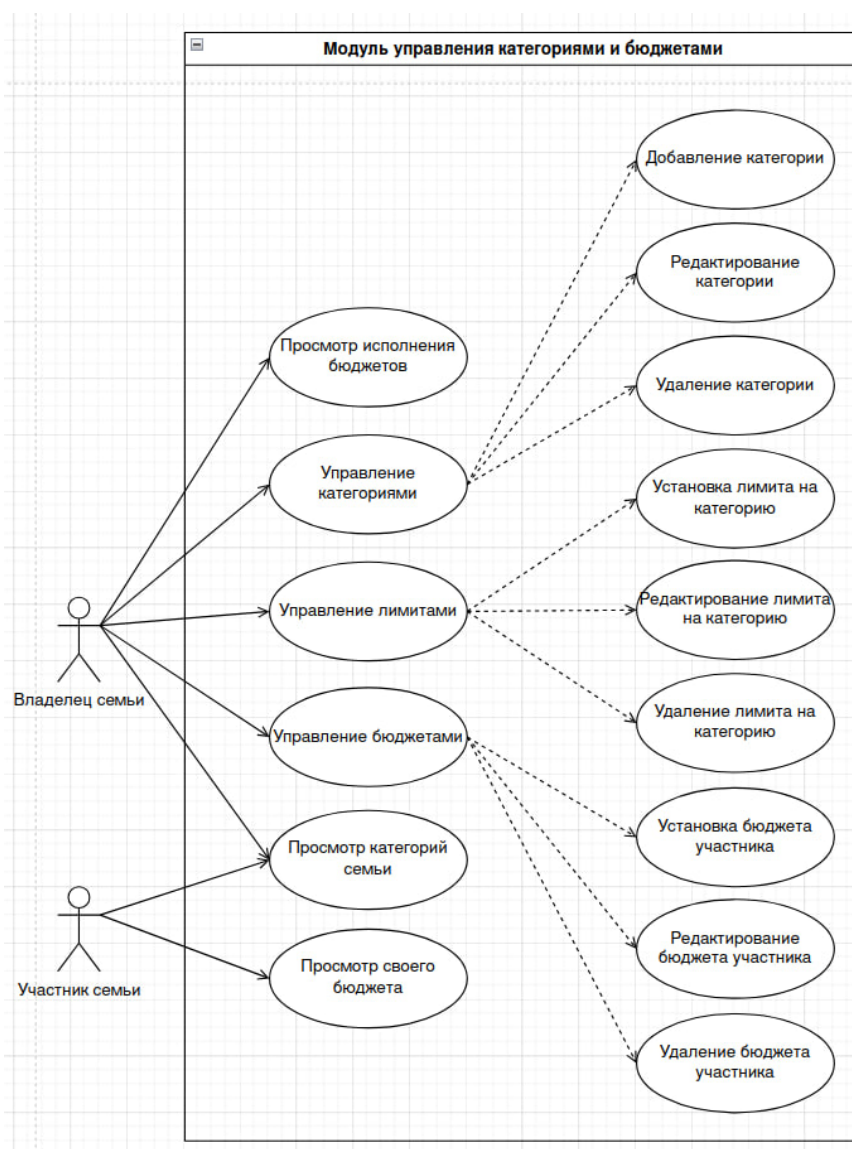


Рисунок 2.3 – Диаграмма вариантов использования модуля управления категориями и бюджетами

Модуль работы с файлами (чеки и отчёты)

Модуль работы с файлами обеспечивает прикрепление чеков к транзак-

циям для сохранения подтверждающих документов, а также экспорт отчётов в различных форматах. В таблице 2.6 приведено распределение прав.

Таблица 2.6 – Действия акторов в модуле работы с файлами

Актор	Доступные действия
Родитель	Загрузка чека к любой транзакции, просмотр чека (любого), удаление чека (любого), экспорт отчёта (CSV, Excel, PDF)
Ребёнок	Загрузка чека к своей транзакции, просмотр своего чека, удаление своего чека
Администратор	Просмотр всех загруженных файлов, удаление любого чека



Рисунок 2.4 – Диаграмма вариантов использования модуля работы с файлами

Модуль аналитики и визуализации

Модуль аналитики и визуализации предоставляет наглядное отображение финансовой информации в виде диаграмм и дашбордов. Это ключевой модуль

для быстрого понимания структуры расходов и доходов семьи. В таблице 2.7 описаны действия акторов.

Таблица 2.7 – Действия акторов в модуле аналитики и визуализации

Актор	Доступные действия
Родитель	Просмотр общего дашборда (доходы, расходы, остаток), просмотр расходов по категориям (круговая диаграмма), просмотр динамики доходов/расходов по месяцам (столбчатая диаграмма), просмотр расходов по участникам семьи, просмотр дашбордов каждого участника
Ребёнок	Просмотр своего дашборда (свои доходы/расходы), просмотр своих расходов по категориям (круговая диаграмма), просмотр остатка карманных денег

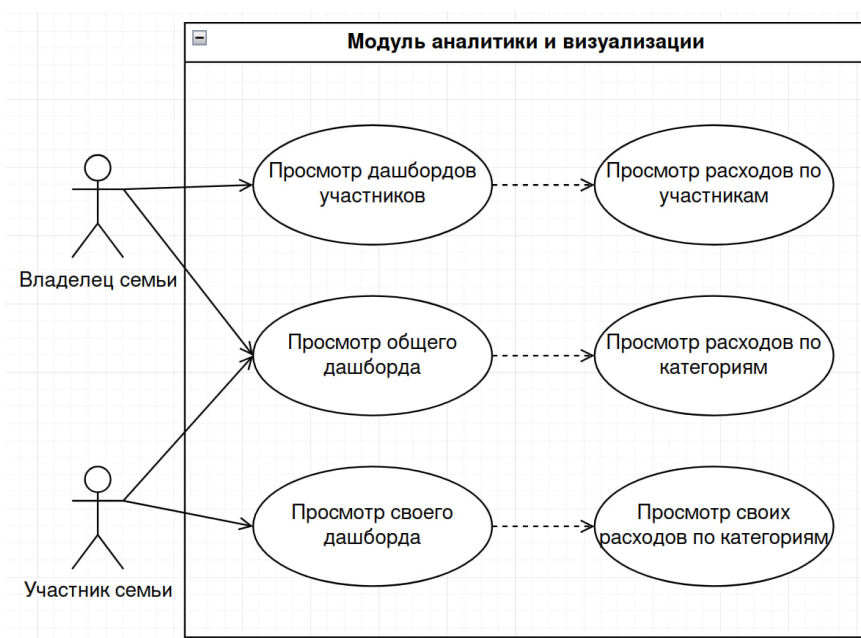


Рисунок 2.5 – Диаграмма вариантов использования модуля аналитики и визуализации

Модуль технического администрирования

Модуль технического администрирования доступен только актору «Администратор (разработчик)» и предназначен для обслуживания, мониторинга и настройки системы. В таблице 2.8 перечислены его действия.

Таблица 2.8 – Действия администратора в модуле технического администрирования

Актор	Доступные действия
Администратор	Вход в админ-панель, просмотр всех пользователей, просмотр всех транзакций (для отладки), просмотр системных логов, резервное копирование базы данных, восстановление базы данных из резервной копии

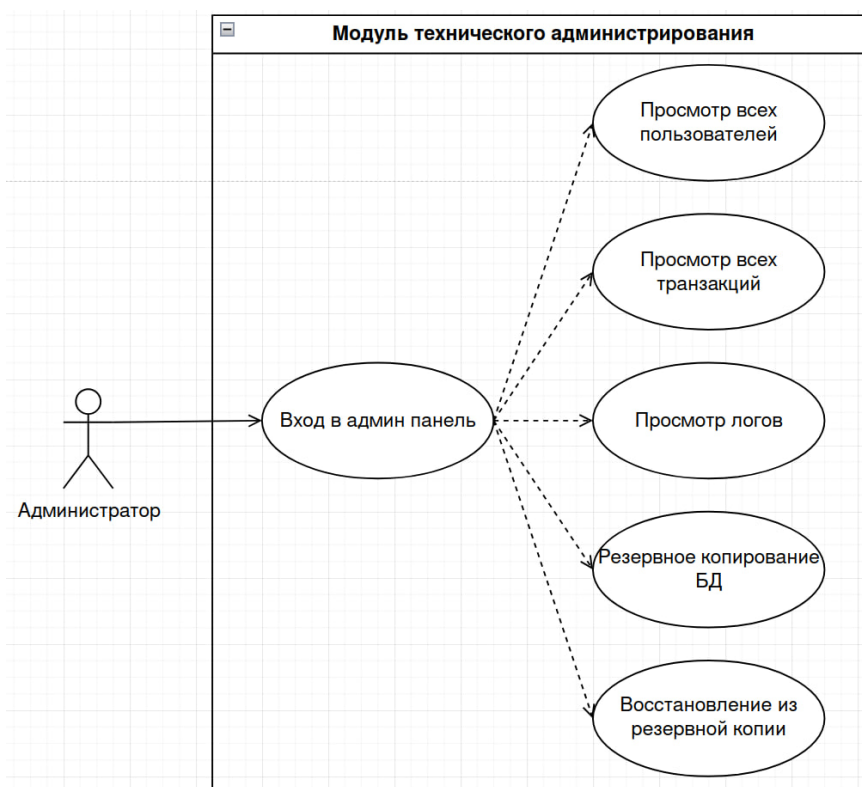


Рисунок 2.6 – Диаграмма вариантов использования модуля технического администрирования

2.2.2 Нефункциональные требования

Нефункциональные требования определяют критерии оценки качества работы системы: производительность, безопасность, удобство использования и надёжность.

1. Требования к производительности

- Время загрузки любой страницы приложения не должно превышать 2 секунд при стандартной нагрузке (до 10 одновременно работающих пользователей).

- Построение диаграмм и агрегация статистики должны выполняться не дольше 1 секунды при объёме до 1000 транзакций в базе данных.
- Система должна корректно обрабатывать одновременные запросы от нескольких членов одной семьи.

2. Требования к безопасности

- Пароли пользователей хранятся в хэшированном виде (bcrypt).
- Запросы к базе данных выполняются через ORM (Flask-SQLAlchemy), что исключает SQL-инъекции.
- При развёртывании на хостинге используется HTTPS для защиты передаваемых данных.

3. Требования к надёжности

- База данных PostgreSQL раз в сутки сохраняется в резервную копию. Администратор может восстановить данные из последней копии.
- Операции, изменяющие несколько сущностей (например, удаление категории с переназначением транзакций), выполняются в рамках транзакции. При сбое все изменения откатываются.
- Система ведёт журнал событий (логи) для отслеживания критических ошибок и подозрительной активности.

4. Требования к удобству использования

- Вёрстка адаптивна и корректно отображается на десктопах (1920×1080, 1366×768), планшетах (768×1024) и смартфонах (375×667 и выше).
- При удалении транзакции, категории или бюджета появляется диалог подтверждения.
- После выполнения любого действия пользователь видит уведомление об успехе или ошибке.

- Интерфейс понятен как родителю, так и ребёнку: чёткие подписи, единообразное расположение элементов, подсказки-плейсхолдеры, цветовое кодирование (зелёный — доход, красный — расход).

Таким образом, описанный набор функциональных модулей полностью покрывает потребности целевой аудитории, обеспечивая совместный учёт, ролевую модель, хранение чеков и наглядную аналитику.

2.3 Проектирование модели данных

На основе анализа предметной области и функциональных требований была разработана логическая модель базы данных. Модель построена с использованием реляционного подхода и нормализована до третьей нормальной формы (3НФ).

Разработанная модель включает восемь основных сущностей, описанных ниже.

Сущность FAMILY (семья) представляет собой группу пользователей, объединённых общим бюджетом. Каждая семья имеет уникальное название и код-приглашение, который используется для добавления новых участников без необходимости вручную настраивать права доступа.

Сущность USER (пользователь) хранит информацию о зарегистрированных пользователях. Каждый пользователь привязан к определённой семье и имеет роль, определяющую его права доступа в системе.

Сущность CATEGORY (категория) предназначена для классификации доходов и расходов. Каждая категория принадлежит определённой семье, что позволяет разным семьям иметь различные наборы категорий.

Сущность TRANSACTION (транзакция) является центральной сущностью приложения и фиксирует каждую операцию поступления или списания средств.

Сущность CATEGORY_LIMIT (лимит на категорию) предназначена для установки ограничений на расходы по определённым категориям для всей семьи. Например, родитель может установить лимит «На категорию "Кафе" тратить не более 5000 рублей в месяц». Данный лимит распространяется на всех членов семьи.

Сущность USER_BUDGET (бюджет участника) предназначена для установки персональных бюджетов для отдельных участников семьи (чаще все-

го — для детей). Родитель может установить ребёнку лимит карманных денег на месяц, который ребёнок не должен превышать. В отличие от CATEGORY_LIMIT, данный бюджет привязан к конкретному пользователю, а не к категории.

Сущность RECEIPT (чек) предназначена для хранения файлов чеков, прикрепляемых к транзакциям. Файлы физически сохраняются на сервере, а в базе данных хранятся пути к ним.

Сущность DASHBOARD_STATS (кэш статистики дашборда) предназначена для хранения предварительно вычисленных агрегированных показателей для дашборда. Данные в этой таблице пересчитываются при изменении транзакций.

На рисунке представлена ER-диаграмма базы данных. Связи между сущностями имеют следующую семантику:

- **FAMILY и USER** — связь «один ко многим»: одна семья может содержать нескольких пользователей, каждый пользователь принадлежит ровно одной семье.
- **FAMILY и CATEGORY** — связь «один ко многим»: одна семья может создавать произвольное количество категорий, каждая категория принадлежит одной семье.
- **USER и TRANSACTION** — связь «один ко многим»: один пользователь может создавать множество транзакций, каждая транзакция привязана к одному пользователю.
- **CATEGORY и TRANSACTION** — связь «один ко многим»: одна категория может использоваться во множестве транзакций, каждая транзакция относится к одной категории.
- **CATEGORY и CATEGORY_LIMIT** — связь «один ко многим»: для одной категории может быть установлено несколько лимитов на разные периоды, каждый лимит привязан к одной категории.
- **USER и USER_BUDGET** — связь «один ко многим»: для одного пользователя может быть установлено несколько бюджетов на разные периоды, каждый бюджет привязан к одному пользователю.

- **TRANSACTION и RECEIPT** — связь «один к одному»: одна транзакция может иметь не более одного чека, каждый чек привязан ровно к одной транзакции.
- **FAMILY и DASHBOARD_STATS** — связь «один ко многим»: одна семья может иметь множество сохранённых статистик для разных периодов, каждая запись статистики принадлежит ровно одной семье.

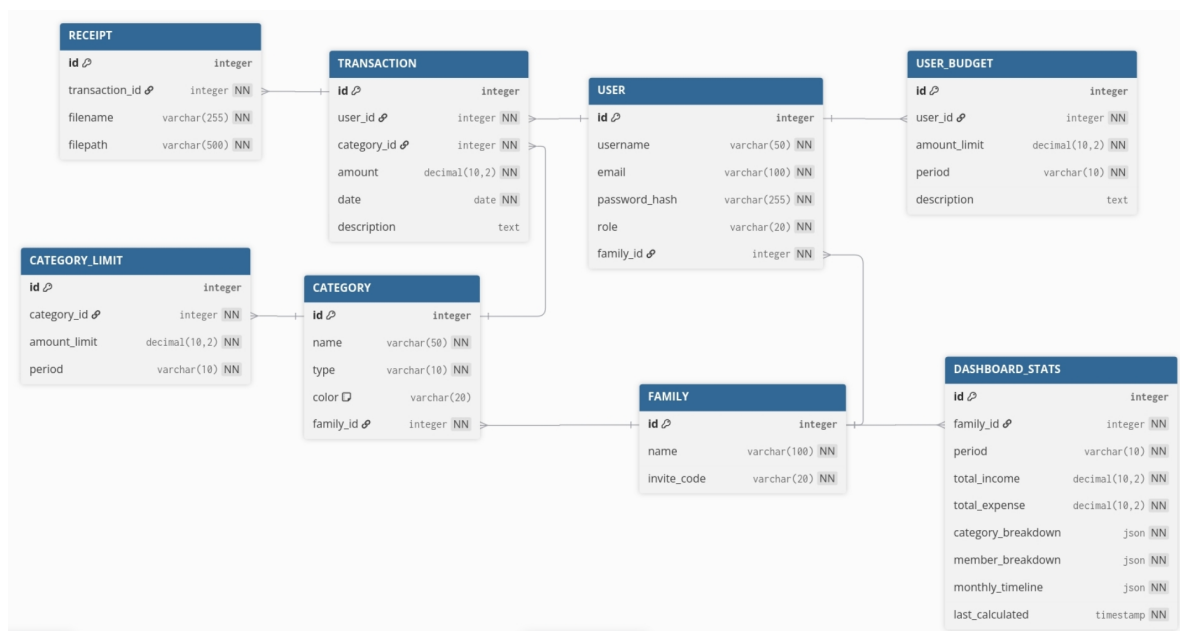


Рисунок 2.7 – ER-диаграмма базы данных веб-приложения для ведения семейного бюджета

2.4 Разработка макетов страниц

На основе анализа целевой аудитории и описанной функциональности были разработаны макеты страниц (wireframes) веб-приложения. Wireframes представляют собой схематичное изображение интерфейса без детальной проработки дизайна, что позволяет сосредоточиться на структуре, навигации и расположении ключевых элементов.

Принципы разработки макетов При создании макетов учитывались следующие принципы:

- **Минимализм** — на каждой странице размещены только необходимые элементы;
- **Единообразие** — все страницы используют единую структуру с хедером, основным содержимым и футером;

- **Ролевая дифференциация** — интерфейс адаптируется под роль пользователя (администратор, владелец, участник).

Все страницы приложения (за исключением страниц входа и регистрации) содержат единую навигационную панель (хэдер), обеспечивающую быстрый переход между разделами. Состав пунктов меню зависит от роли пользователя.

Страница входа

Макет страницы входа содержит хэдер со ссылкой на страницу регистрации, форму авторизации с полями «Логин» и «Пароль», кнопку «Вход» для отправки формы, а также футер. Страница служит точкой входа в систему для всех категорий пользователей.

The diagram illustrates the layout of the login page. It is a rectangular box divided into three horizontal sections. The top section is a header labeled 'Хэдер' on the left and contains an oval button labeled 'Регистрация' on the right. The middle section is the main content area, which is centered and contains a rounded rectangle labeled 'Форма входа'. Inside this form are two horizontal input fields: the top one is labeled 'Логин' and the bottom one is labeled 'Пароль'. Below these fields is an oval button labeled 'Вход'. The bottom section of the main box is a footer labeled 'Футер'.

Рисунок 2.8 – Макет страницы входа

Страница регистрации

Макет страницы регистрации содержит хэдер со ссылкой на страницу входа. Форма регистрации включает следующие поля: логин, пароль, подтверждение пароля, выбор роли, а также поле «Код подключения», которое отображается при выборе роли «Участник» для присоединения к существующей семье. При выборе роли «Владелец» новая семья создаётся автоматически.

Рисунок 2.9 – Макет страницы регистрации

Главная страница (администратор)

Макет главной страницы для администратора предназначен для технического обслуживания системы. Хэдер содержит выпадающее меню «Управление приложением», раздел «Управление пользователями», а также раздел «Управление файлами и транзакциями».

Рисунок 2.10 – Макет дашборда для администратора

Главная страница (владелец семьи)

Макет главной страницы для владельца семьи содержит фильтр с выбором периода (день/неделя/месяц). На странице отображаются ключевые показатели: расходы за период, доходы за период, бюджет и остаток от бюджета. Визуализация включает динамику бюджета за период (линейный график) и круговую диаграмму распределения расходов по категориям. Также на странице представлены блоки «Лимиты на категории», «Дашборды участников», «Все транзакции за период», «Все категории на период» и кнопка «Сформировать отчёт за период».

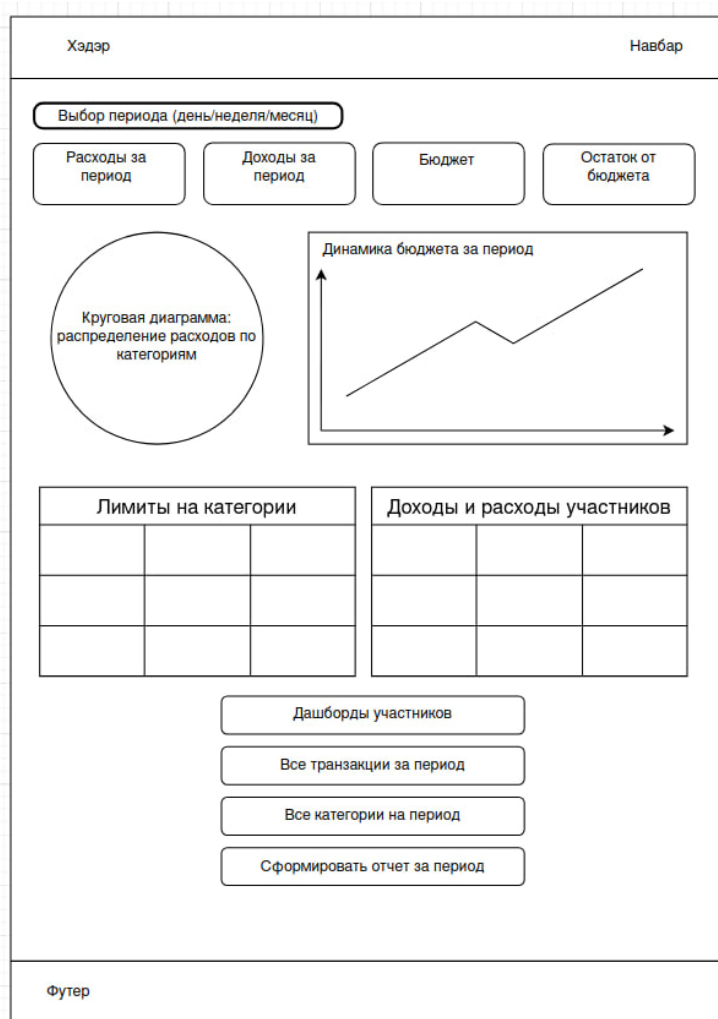


Рисунок 2.11 – Макет дашборда для владельца семьи

Главная страница (участник семьи)

Макет главной страницы для участника является упрощённой версией дашборда владельца. Хэдер содержит выбор периода (день/неделя/месяц). Отображаются личные показатели участника: расходы за период, доходы за период, бюджет и остаток от бюджета. Визуализация идентична с

визуализацией владельца семьи. Дополнительно представлены блоки «Лимиты на категории», «Свои транзакции за период», «Свои категории на период» и кнопка «Сформировать отчет за период».

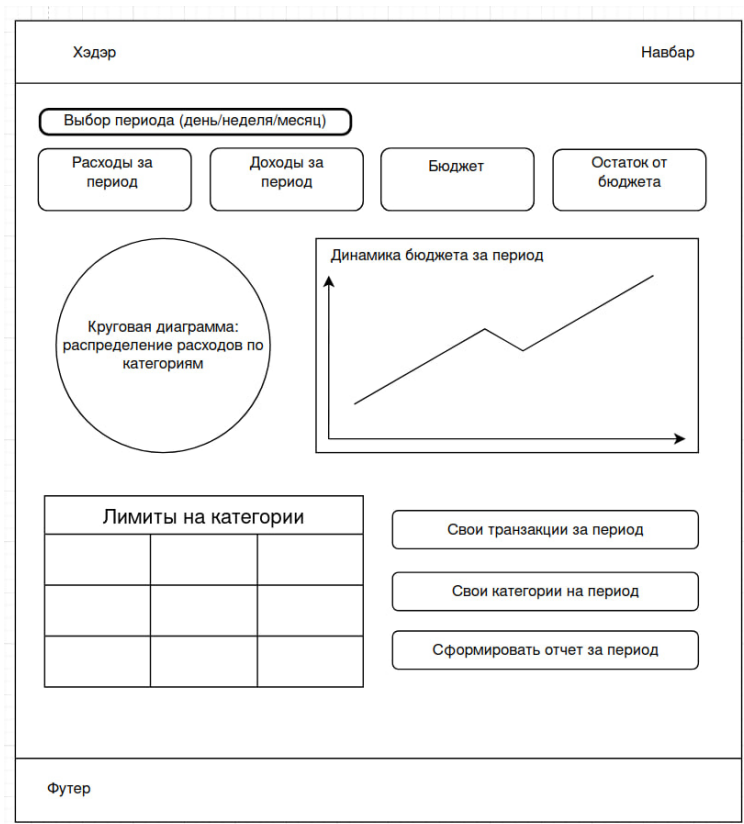


Рисунок 2.12 – Макет дашборда для участника семьи

Страница профиля

Макет страницы профиля содержит раздел «Личные данные» с отображением логина, пароля, роли, названия семьи и текущего бюджета пользователя. Также на странице представлен раздел «Участники семьи», в котором перечислены все члены семьи.

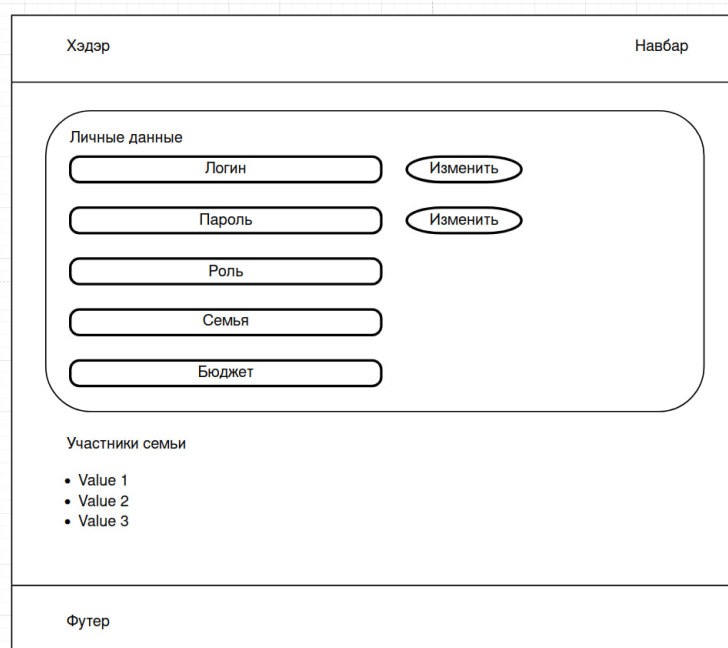


Рисунок 2.13 – Макет страницы профиля

Страница управления участниками (владелец)

Макет страницы управления участниками, доступной только владельцу семьи, содержит два основных блока. Первый блок — «Список всех участников семьи» с возможностью поиска, отображением имени участника, а также кнопками действий: «Подробнее», «Установить бюджет», «Удалить».

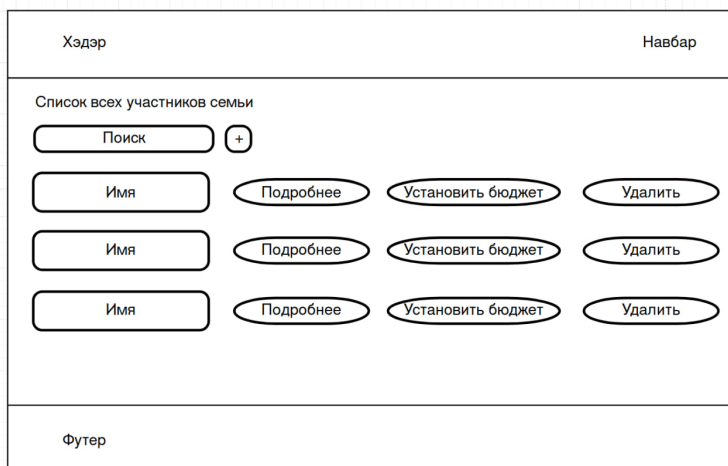


Рисунок 2.14 – Макет страницы управления участниками

Страница управления транзакциями

Макет страницы управления транзакциями предназначен для просмотра и управления всеми финансовыми операциями. Страница информация о транзакции: дата, категория, сумма, участник (для владельца). Предусмотрены возможности поиска и фильтрации транзакций. Для каждой транзакции до-

ступны кнопки действий: «Редактировать», «Удалить», «Добавить чек». Также на странице присутствует кнопка «+» «Добавить транзакцию» для создания новой операции.

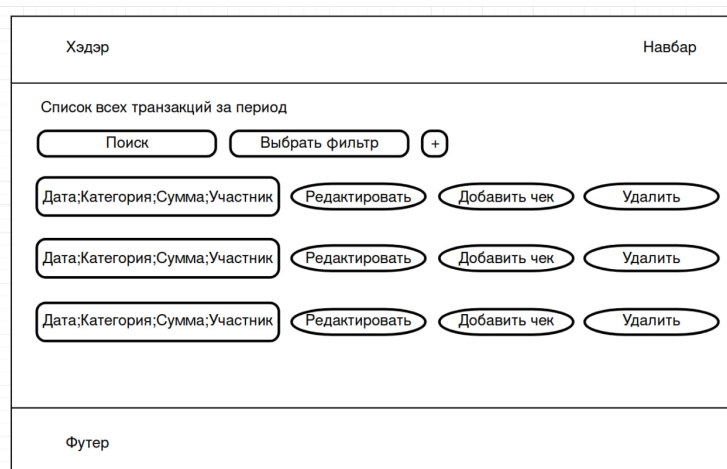


Рисунок 2.15 – Макет страницы управления транзакциями

Страница управления категориями (владелец)

Макет страницы управления категориями, доступной только владельцу семьи, содержит таблицу всех категорий с колонками: «Имя», «Описание», а также кнопками действий: «Редактировать», «Установить лимит», «Удалить». Предусмотрена возможность поиска категорий по названию.

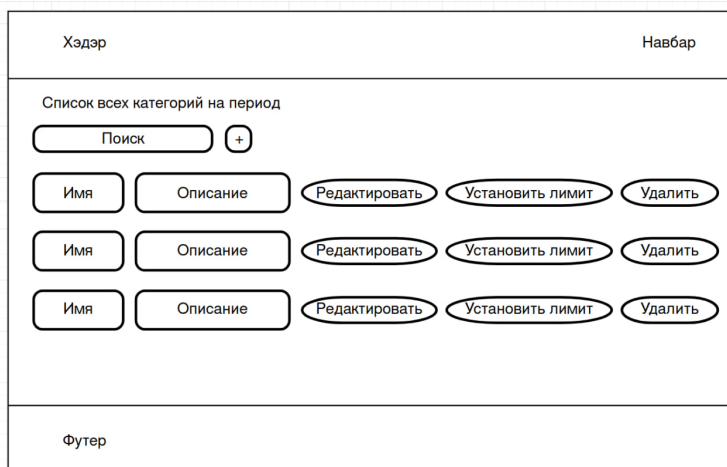


Рисунок 2.16 – Макет страницы управления категориями

Разработанные wireframes полностью покрывают функциональные требования. Все макеты учитывают разграничение прав между администратором, владельцем семьи и участником.

ГЛАВА 3. РАЗРАБОТКА ВЕБ-ПРИЛОЖЕНИЯ

В соответствии с результатами проектирования было разработано веб-приложение для ведения семейного бюджета.

3.1 Вёрстка HTML страниц

Разработка пользовательского интерфейса началась с создания базового шаблона **base.html**, который определяет общую структуру всех страниц приложения. Шаблон включает:

- заголовок страницы с динамическим заголовком;
- подключение Bootstrap 5.3 (CSS и JS) через CDN;
- подключение кастомных стилей **style.css**;
- навигационную панель (navbar), содержимое которой зависит от роли пользователя (owner, member, admin);
- блок для отображения flash-сообщений (уведомлений);
- основную область контента (блок **content**);
- футер с копирайтом.

На основе базового шаблона были свёрстаны следующие страницы:

- **auth/login.html** — страница входа;
- **auth/register.html** — страница регистрации (выбор роли: владелец/участник, поле email и кода семьи);
- **dashboard/owner_dashboard.html** — дашборд владельца семьи;
- **dashboard/member_dashboard.html** — дашборд участника семьи;
- **dashboard/admin_dashboard.html** — панель администратора;
- **dashboard/member_stats.html** — страница просмотра статистики участника (для владельца);
- **transactions.html** — управление транзакциями с фильтрацией;

- **categories.html** — управление категориями;
- **members.html** — управление участниками семьи;
- **profile.html** — страница профиля пользователя;
- **admin/all_transactions.html** — просмотр всех транзакций (админ);
- **admin/all_files.html** — просмотр всех загруженных файлов (админ).

3.2 Реализация модуля авторизации

Модуль авторизации реализован с использованием расширений **Flask-Login** (управление сессиями) и **bcrypt** (хэширование паролей) через методы **set_password** и **check_password** в модели **User**.

Регистрация пользователя реализована в маршруте **/auth/register** с учётом ролевой модели:

- При выборе роли «**Владелец**» автоматически создаётся новая семья с названием «Семья {username}» и генерируется 6-значный код-приглашение (формата FAM-XXXXXX). Пользователь становится владельцем семьи. Также автоматически создаются 8 категорий по умолчанию: Продукты, Транспорт, Коммунальные услуги, Зарплата, Подработка, Кафе и рестораны, Развлечения, Одежда.
- При выборе роли «**Участник**» пользователь должен ввести код-приглашение существующей семьи. Система проверяет корректность кода и добавляет пользователя в соответствующую семью с ролью **'member'**.

Аутентификация выполняется через маршрут **/auth/login**. После успешного входа пользователь перенаправляется на дашборд, соответствующий его роли. Доступ к страницам контролируется декоратором **@login_required**, а для разграничения прав используется проверка **current_user.role**. Для администратора создана учётная запись по умолчанию (логин **'admin'**, пароль **'admin123'**), которая создаётся автоматически при инициализации базы данных.

3.3 Реализация CRUD-интерфейса

CRUD-интерфейс (Create, Read, Update, Delete) реализован для трёх основных сущностей: транзакции, категории и участники. Все операции выполняются через веб-формы с валидацией данных на стороне сервера.

Управление транзакциями

Для работы с транзакциями создана модель **Transaction**. Связь с чеком реализована через модель **Receipt**. Реализованы следующие маршруты и API-эндпоинты:

- **GET /transactions** — отображение списка транзакций с фильтрацией по поисковому запросу и диапазону дат;
- **POST /add_transaction** — добавление транзакции (сумма автоматически становится отрицательной для расходов);
- **POST /edit_transaction** — редактирование транзакции;
- **POST /api/delete_transaction/<int:id>** — удаление транзакции;
- **GET /api/get_transaction/<int:id>** — получение данных транзакции для редактирования.

При добавлении транзакции сумма вводится как положительное число, затем на основе типа категории определяется знак: для **'expense'** сумма становится отрицательной, для **'income'** — положительной. Пользователь с ролью **'owner'** может добавлять транзакции для любого члена семьи, участник — только для себя.

Управление категориями

Для реализации CRUD-функционала категорий создана модель **Category**. Категории доступны только для владельца семьи через маршрут **/categories**. Владелец может создавать, редактировать и удалять категории. При удалении категории все связанные с ней транзакции и чеки также удаляются для сохранения целостности данных.

Для установки лимитов на категории используется модель **CategoryLimit**. Установка лимита реализована через маршрут **/set_category_limit**. На странице категорий отображается текущий лимит для каждой категории (если установлен).

Управление участниками

Управление участниками семьи доступно владельцу через маршрут **/members**. На странице отображается список всех членов семьи с их ролями, email и установленным бюджетом. Реализованы следующие возможности:

- **Просмотр** — отображение всех участников с поиском по имени;
- **Установка бюджета** — владелец может установить персональный бюджет для участника через маршрут `/set_user_budget` (модель `UserBudget`);
- **Удаление участника** — через API-эндпоинт `/api/delete_member/<int:id>` (все транзакции и чеки участника также удаляются);
- **Генерация кода приглашения** — через маршрут `/generate_invite_code`.

Все операции защищены проверкой роли: доступ только для `'owner'`.

3.4 Реализация обработки и анализа данных

Обработка и анализ данных реализованы на уровне маршрутов и моделей. Для формирования аналитических дашбордов в приложении реализован механизм агрегации транзакционных данных. Основные задачи агрегации:

1. фильтрация транзакций по выбранному периоду (день/неделя/месяц);
2. группировка расходов по категориям для круговой диаграммы;
3. расчёт накопленного баланса для отображения динамики.

Для оптимизации производительности используется модель `DashboardStats`, которая кэширует ежедневные балансы пользователей в поле `daily_balance` (тип `JSON`). Функция `update_dashboard_stats(user_id)` пересчитывает балансы после каждого изменения транзакций.

Листинг 1 – Листинг 3.1 — Фильтрация транзакций по периоду и расчёт доходов

```

1 def get_transactions(self, period='all'):
2     """Получить транзакции пользователя за период"""
3     now = datetime.utcnow()
4
5     if period == 'day':
6         start_date = now.replace(hour=0, minute=0,
7                                   second=0, microsecond=0)
8         end_date = now.replace(hour=23, minute=59,
9                                 second=59, microsecond=999999)
10    elif period == 'week':

```

```

9         start_date = now - timedelta(days=now.weekday())
10        start_date = start_date.replace(hour=0, minute=0,
second=0, microsecond=0)
11        end_date = now.replace(hour=23, minute=59,
second=59, microsecond=999999)
12        else: # month
13            start_date = now.replace(day=1, hour=0, minute=0,
second=0, microsecond=0)
14            if now.month == 12:
15                end_date = now.replace(year=now.year+1,
month=1, day=1) - timedelta(seconds=1)
16            else:
17                end_date = now.replace(month=now.month+1,
day=1) - timedelta(seconds=1)
18
19        return
Transaction.query.filter_by(user_id=self.id).filter(
20            Transaction.date >= start_date,
21            Transaction.date <= end_date
22        ).all()
23
24    def get_total_income(self, period='all'):
25        """Получить сумму доходов за период"""
26        transactions = self.get_transactions(period)
27        return sum(float(t.amount) for t in transactions if
t.amount > 0)

```

Метод **get_transactions()** определяет временные границы в зависимости от выбранного пользователем периода (день, неделя или месяц). Для периода «день» устанавливаются границы с 00:00 до 23:59 текущего дня, для «недели» — с понедельника по текущий день, для «месяца» — с 1-го числа по текущий день. Метод **get_total_income()** агрегирует сумму всех положительных транзакций (доходов) за указанный период с использованием встроенной функции **sum()** и генератора списков.

Листинг 2 – Листинг 3.2 — Группировка расходов по категориям

```

1 # Группировка расходов по категориям
2 expense_by_category = {}
3 for transaction in family_transactions_all:
4     if transaction.amount < 0: # Только расходы

```

```

5         category_name = transaction.category.name
6         amount = float(abs(transaction.amount))
7         expense_by_category[category_name] =
            expense_by_category.get(category_name, 0.0) + amount
8
9 expense_categories = list(expense_by_category.keys())
10 expense_amounts = list(expense_by_category.values())

```

Алгоритм выполняет обход всех транзакций семьи за период, отбирает только расходы (отрицательные суммы) и накапливает суммы в словаре **expense_by_category**, где ключом выступает название категории. На выходе формируются два списка: названия категорий и соответствующие суммы для последующей визуализации в круговой диаграмме Chart.js.

Листинг 3 – Листинг 3.3 — Расчёт накопленного баланса

```

1 def update_dashboard_stats(user_id):
2     """Обновление статистики для дашборда пользователя"""
3     # Получаем все транзакции семьи
4     all_transactions = Transaction.query.filter(
5         Transaction.user_id.in_(family_member_ids)
6     ).order_by(Transaction.date.asc()).all()
7
8     # Вычисляем баланс на каждый день накопленный( итог)
9     daily_balance = {}
10    running_balance = 0.0
11
12    while current_date <= end_date:
13        # Суммируем доходы и расходы за день
14        day_income = sum(float(t.amount) for t in
15            day_transactions if t.amount > 0)
16        day_expense = sum(float(t.amount) for t in
17            day_transactions if t.amount < 0)
18        running_balance += (day_income + day_expense)
19
20        daily_balance[current_date.isoformat()] =
21            running_balance
22        current_date += timedelta(days=1)

```

Накопленный баланс вычисляется последовательным суммированием ежедневных доходов и расходов. Алгоритм последовательно обрабаты-

ет дни от первой транзакции до текущей даты, поддерживая переменную **running_balance**, которая хранит баланс на конец каждого дня. Значение баланса на конец каждого дня сохраняется в кэше (JSON-поле **daily_balance** таблицы **DashboardStats**) для оптимизации повторных запросов.

Агрегация данных для дашбордов

Для дашборда владельца семьи (**/dashboard/owner**) реализована следующая логика:

- расчёт общих доходов и расходов семьи за выбранный период (день/неделя/месяц);
- расчёт семейного бюджета на основе суммы персональных бюджетов всех участников (с приведением к выбранному периоду);
- группировка расходов по категориям для круговой диаграммы;
- построение столбчатой диаграммы динамики баланса на основе кэшированных данных из **DashboardStats**;
- сводная таблица по участникам с доходами, расходами и балансом.

Для дашборда участника (**/dashboard/member**) реализована аналогичная логика, но только для одного пользователя. Владелец также может просматривать дашборд любого участника через маршрут **/member_stats/<int:member_id>**.

Формирование отчётов

Экспорт отчётов реализован в формате CSV через маршрут **/export_report**. Отчёт включает все транзакции (для владельца — всей семьи, для участника — только свои) с колонками: дата, описание, категория, сумма, тип, участник. Файл автоматически скачивается с именем **report_<period>_<data>.csv**.

3.5 Реализация работы с файлами

Загрузка и хранение чеков

При добавлении или редактировании транзакции пользователь может загрузить файл чека. Реализована следующая логика:

- Допустимые форматы: изображения (.jpg, .jpeg, .png) и PDF;
- Максимальный размер файла: 16 МБ;

- Файл сохраняется в папке **uploads/** с использованием **secure_filename**;
- В таблице **Receipt** сохраняется путь к файлу и связь с транзакцией;
- При удалении транзакции связанный файл чека также удаляется с диска.

Просмотр чека реализован через API-эндпоинт **/api/get_receipt/<int:id>**, который возвращает URL файла для отображения в модальном окне.

Администратор имеет доступ к просмотру всех файлов через страницу **/admin/view_all_files**.

3.6 Реализация системы визуализации

Для наглядного отображения финансовой аналитики используется библиотека Chart.js. Визуализация реализована на дашбордах владельца и участника.

Круговая диаграмма расходов по категориям

Отображает структуру расходов за выбранный период. Данные передаются через переменные **expense_categories** и **expense_amounts**. Диаграмма строится с типом **pie** с цветовой палитрой из 10 цветов. При наведении на сектор отображается всплывающая подсказка с названием категории, суммой и процентным соотношением.

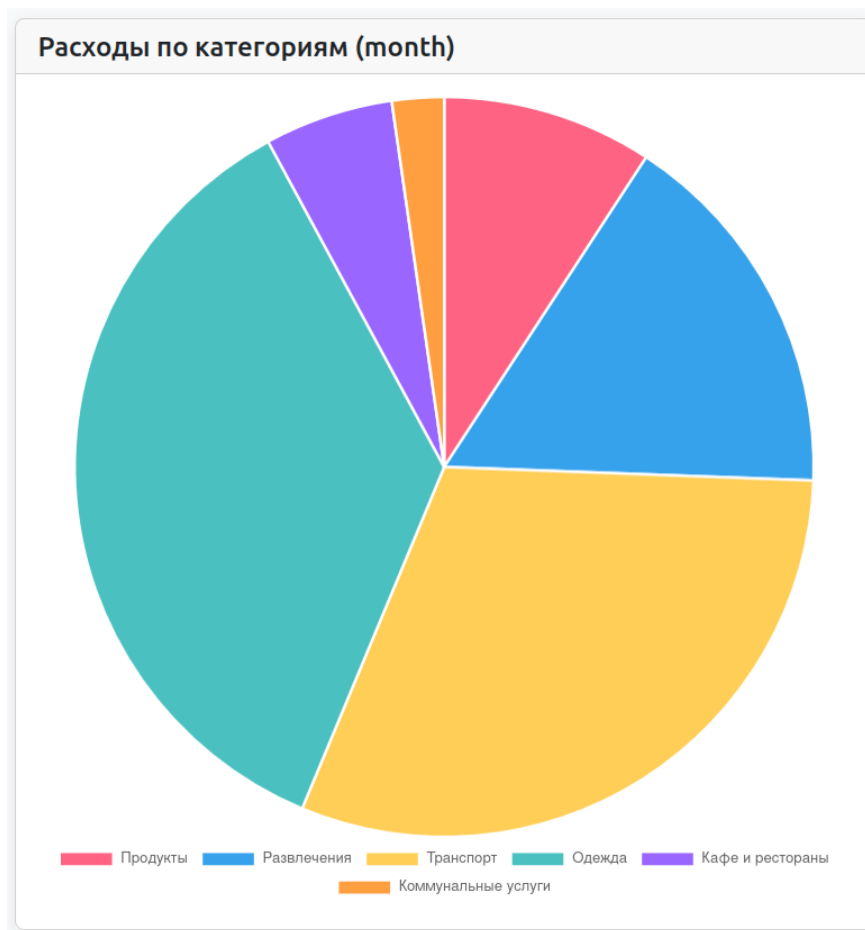


Рисунок 3.1 – Круговая диаграмма расходов по категориям

Столбчатая диаграмма динамики баланса

Показывает изменение баланса по дням выбранного периода. Данные передаются через переменные **chart_labels** (метки дат) и **chart_balance_data** (значения баланса). Для владельца диаграмма имеет цветовое кодирование: зелёные столбцы для дней с положительным балансом, красные — для отрицательных, серые — для будущих дней. Диаграмма строится с типом **bar**.

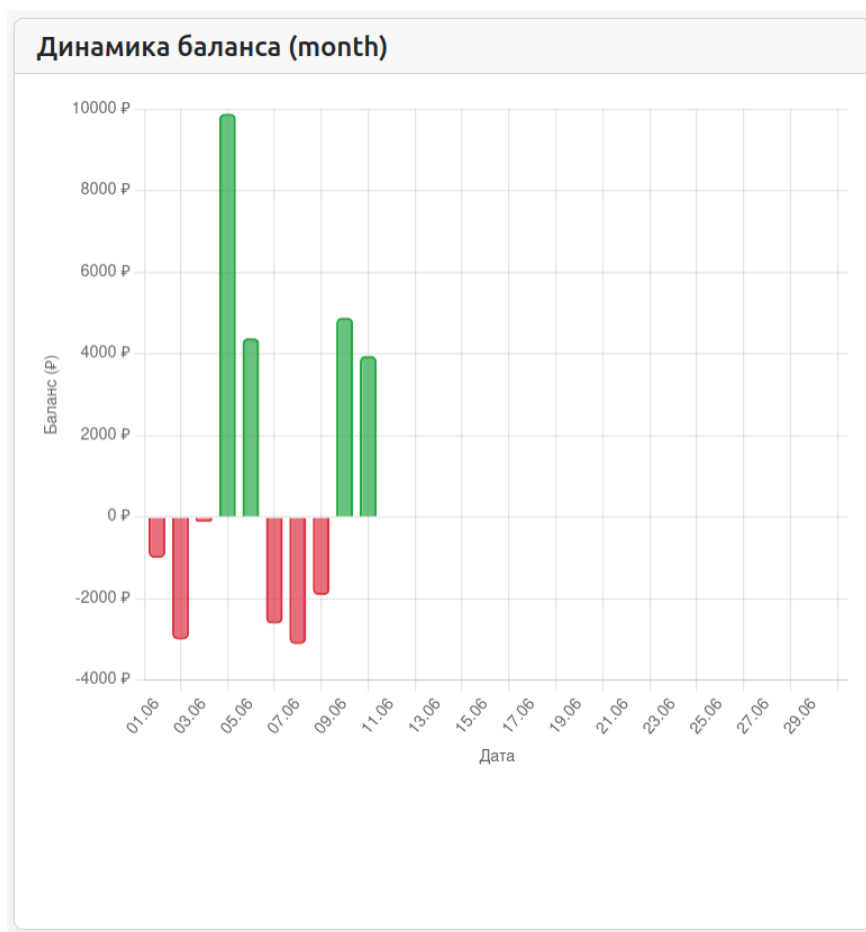


Рисунок 3.2 – Столбчатая диаграмма динамики баланса

Сводная таблица по участникам

На дашборде владельца реализована таблица с агрегированными данными по каждому участнику семьи: доходы, расходы, баланс. Для каждого участника предусмотрена кнопка «Дашборд», открывающая детальную статистику участника.

Участник	Роль	Доходы (₽)	Расходы (₽)	Баланс (₽)	Действия
user1	Участник	8,000 ₽	8,000 ₽	0 ₽	Дашборд
user2	Участник	2,500 ₽	3,700 ₽	-1,200 ₽	Дашборд
user3	Участник	3,000 ₽	800 ₽	2,200 ₽	Дашборд
owner Это вы	Владелец	10,000 ₽	7,044 ₽	2,956 ₽	Дашборд

Рисунок 3.3 – Сводная таблица по участникам семьи

Все диаграммы автоматически перестраиваются при изменении периода через параметр URL **?period=**.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсового проекта было разработано веб-приложение для ведения семейного бюджета **MoneyManage**. Приложение позволяет семьям совместно контролировать доходы и расходы, устанавливать бюджеты для участников, загружать чеки и получать наглядную статистику.

В результате выполнения работы были решены все поставленные задачи:

1. Проведён анализ существующих программных продуктов (Дзен-мани, CoinKeeper, Money Lover), выявивший их недостатки: платные премиум-функции, слабая поддержка совместной работы, отсутствие полноценных веб-версий. Обоснован выбор технологического стека: Python + Flask, PostgreSQL, Bootstrap 5, Chart.js.
2. Определены требования целевой аудитории (семьи с совместным бюджетом) и спроектирована функциональность приложения, включая диаграммы вариантов использования и User Story.
3. Разработана модель данных, включающая 8 сущностей, построена ER-диаграмма.
4. Созданы макеты страниц (wireframes) для всех категорий пользователей: администратора, владельца семьи и участника.
5. Реализована система аутентификации и авторизации с тремя ролями (**admin**, **owner**, **member**) с использованием **Flask-Login** и **bcrypt**. При регистрации владельца автоматически создаются категории по умолчанию.
6. Разработан CRUD-интерфейс для транзакций, категорий и участников. Реализованы фильтрация транзакций по дате и поиску, установка лимитов на категории и персональных бюджетов.
7. Реализованы механизмы загрузки чеков (изображения, PDF) с сохранением на сервере и хранением пути в БД. Добавлен экспорт отчётов в формате CSV.
8. Реализована система визуализации: круговая диаграмма расходов по категориям и столбчатая диаграмма динамики баланса (Chart.js). Для оп-

тимизации производительности используется кэширование ежедневных балансов в модели **DashboardStats**.

9. Выполнен деплой приложения на хостинг Railway.com. Приложение доступно по адресу: <https://awake-unity-production-32a1.up.railway.app>. Проведено тестирование работоспособности развёрнутого приложения.
10. Код приложения и отчетная документация Latex размещены в репозитории GitHub: https://github.com/userownmaa/web_coursework_Momina_241-327.

Разработанное приложение представляет собой полноценный программный продукт, готовый к развёртыванию на хостинге. Дальнейшее развитие проекта может включать: добавление мобильной версии, интеграцию с банковскими API для автоматического импорта транзакций, расширенную аналитику с прогнозированием расходов, а также улучшение системы уведомлений о превышении лимитов.

СПИСОК ЛИТЕРАТУРЫ

- [1] Индекс финансовой грамотности — 2025: россияне стали хуже считать проценты [Электронный ресурс], Аналитический центр НАФИ. — [Б. м. : б. и.], 2026. — URL: <https://nafi.ru/polls/indeks-finansovoy-gramotnosti-2025-rossiyane-stali-khuzhe-schitat-protsenty/> (дата обращения: 01.04.2026).
- [2] Банк России принял решение снизить ключевую ставку [Электронный ресурс], Центральный банк Российской Федерации. — [Б. м. : б. и.], 2026. — 03. — URL: https://www.cbr.ru/press/pr/?file=20032026_133000key.htm (дата обращения: 01.04.2026).
- [3] Дзен-мани: Официальный сайт приложения для учёта личных финансов [Электронный ресурс], Дзен-мани. — [Б. м. : б. и.], 2026. — URL: <https://zenmoney.ru/> (дата обращения: 01.04.2026).
- [4] Monefy — Budget Tracker App [Electronic resource], Reflective Technologies ApS. — [S. l. : s. n.], 2026. — URL: <https://monefy.com/> (online; accessed: 01.04.2026).
- [5] Money Lover – Spending Manager [Electronic resource], Google Play. — [S. l. : s. n.], 2026. — URL: <https://play.google.com/store/apps/details?id=com.bookmark.money> (online; accessed: 01.04.2026).
- [6] Django Documentation [Electronic resource], Django Software Foundation. — [S. l. : s. n.], 2026. — URL: <https://docs.djangoproject.com/> (online; accessed: 01.04.2026).
- [7] Welcome to Flask — Flask Documentation (3.1.x) [Electronic resource], Flask. — [S. l. : s. n.], 2026. — URL: <https://flask.palletsprojects.com/> (online; accessed: 01.04.2026).
- [8] FastAPI Framework Official Documentation [Electronic resource], FastAPI. — [S. l. : s. n.], 2026. — URL: <https://fastapi.tiangolo.com/> (online; accessed: 01.04.2026).
- [9] Bootstrap 5 JavaScript Documentation [Electronic resource], Bootstrap. — [S.

- l. : s. n.], 2026. — URL: <https://getbootstrap.com/docs/5.3/getting-started/javascript/> (online; accessed: 01.04.2026).
- [10] React Documentation: Quick Start [Электронный ресурс]. — [Б. м. : б. и.]. — URL: <https://react.dev> (дата обращения: 27.03.2026).
- [11] Chart.js | Open source HTML5 Charts for your website [Electronic resource], Chart.js. — [S. l. : s. n.], 2026. — URL: <https://www.chartjs.org/> (online; accessed: 01.04.2026).